

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МОЭВМ**

**ОТЧЕТ**  
**по лабораторной работе №2**  
**по дисциплине «Объектно-ориентированное программирование»**

Студент гр. 4385

\_\_\_\_\_

Юферев А.С.

Преподаватель

\_\_\_\_\_

Жангиров Т.Р.

Санкт-Петербург

2025

### **Цель работы:**

Расширение функциональности пошаговой игры с использованием принципов объектно-ориентированного программирования, реализация механики заклинаний, динамического управления ресурсами и взаимодействия объектов.

### **Задача:**

В рамках лабораторной работы необходимо создать:

1. Иерархию классов заклинаний с использованием виртуальных функций.
2. Базовый абстрактный класс Spell.
3. Производные классы:
  - DamageSpell (прямой урон),
  - AreaSpell (урон по области).
4. Класс Hand для хранения заклинаний (динамическое управление памятью).
5. Логику получения нового заклинания после определённого количества убийств.
6. Расширенную обработку ходов игрока и врагов.
7. Корректную реализацию правила пяти для классов, владеющих ресурсами.

### **Выполнение работы:**

Перед началом реализации были выделены основные сущности игры:

- **Cell** – клетка игрового поля
- **GameField** – игровое поле
- **Player** – игрок
- **Enemy** – враг
- **Spell** – абстрактное заклинание
- **DamageSpell** – заклинание прямого урона
- **AreaSpell** – заклинание урона по области

- **Hand** – контейнер заклинаний игрока
- **Game** – класс управления игровым процессом

Рисунок 1 – UML-диаграмма классов игры



## **Описание классов:**

### **Cell:**

- Инкапсулирует состояние клетки поля.
- Реализовано правило пяти.
- Обеспечивается инвариант: клетка не может одновременно содержать игрока и врага.

### **Gamefield:**

- Хранит двумерный вектор клеток.
- Размер поля задаётся через конструктор (от 10 до 25).
- Реализованы:
  - Конструктор копирования (глубокая копия),
  - Конструктор перемещения,
  - Операторы присваивания,
  - Проверка корректности размеров.

### **Enemy:**

- Хранит здоровье, урон и координаты.
- Реализовано автоматическое перемещение к игроку.
- При достижении игрока наносит урон и остаётся на месте.
- После смерти помечается как неживой.

### **Player:**

- Хранит:
  - здоровье,
  - урон,
  - очки,
  - координаты,

- указатель на Hand.
- Реализовано правило пяти (владение динамической памятью).
- После определённого количества убийств получает новое заклинание.

Spell:

- Содержит:
  - имя заклинания,
  - радиус действия,
  - флаг использования за ход.
- Объявлен чисто виртуальный метод use().
- Реализована проверка дальности применения.

DamageSpell:

- Наносит фиксированный урон одной цели.
- Проверяет наличие врага в выбранной клетке.
- При уничтожении врага начисляет очки игроку.

AreaSpell:

- Наносит урон по области (квадрат  $\text{areaSize} \times \text{areaSize}$ ).
- Может поражать нескольких врагов.
- Суммирует нанесённый урон.

**Hand:**

- Хранит вектор указателей на Spell.
- Управляет динамической памятью.
- Реализованы:
  - конструктор копирования (глубокое копирование),
  - оператор присваивания,
  - перемещающий конструктор,
  - деструктор.
- Ограничение максимального количества заклинаний.
- Автоматическое добавление нового заклинания.

Game:

- Координирует взаимодействие всех объектов.
- Реализует:
  - основной игровой цикл,
  - режим заклинаний,
  - обработку ввода,
  - поочерёдные ходы,
  - проверку победы и поражения.

### Результаты тестирования:

| Действие                      | Ожидаемый результат         | Результат |
|-------------------------------|-----------------------------|-----------|
| Размер поля 5×5               | Сообщение об ошибке         | Пройдено  |
| Размер поля 20×20             | Успешное создание           | Пройдено  |
| Уничтожение врага заклинанием | Удаление с поля +10 очков   | Пройдено  |
| Убийство 3 врагов             | Получение нового заклинания | Пройдено  |
| Смерть игрока                 | Сообщение о поражении       | Пройдено  |
| Уничтожение всех врагов       | Сообщение о победе          | Пройдено  |

Таблица 1 – тестирование

### Рисунок 2 – Запуск игры

```
C:\Users\vc\Desktop\OOP\1b2>a.exe
=== Добро пожаловать в игру! ===
Введите размер поля (ширина высота) от 10 до 25:
```

### Рисунок 3 – Использование заклинания

```
=== Игрок ===
Здоровье: 1000 | Урон: 10 | Очки: 0
Врагов осталось: 3
Прогресс убийств: 0/3

--- Рука игрока ---
1. Взрывная волна (радиус: 4)

=== РЕЖИМ ЗАКЛИНАНИЙ ===
Цель: (2,5)
Перемещение: w/a/s/d, Enter - применить (потратит ход), ESC - отмена

Проверка дальности:
Игрок: (5,5)
Цель: (2,5)
Цель в радиусе действия

=== ПРИМЕНЕНИЕ ЗАКЛИНАНИЯ ===
Заклинание: Взрывная волна
Центр области: (2,5)
Область поражения: 2x2
Враг в клетке (2,5) получил 15 урона!
Всего поражено врагов: 1
Суммарный урон: 15
=== ЗАКЛИНАНИЕ ПРИМЕНЕНО ===
Нажмите любую клавишу для продолжения...

--- Ход врагов ---
Ход врагов завершен. Нажмите любую клавишу...
```

### Рисунок 4 – Добавление заклинания

```
Атака! Нанесен урон врагу!
Получено очков: 10. Всего: 30
Прогресс убийств: 3/3
=== ПОЛУЧЕНО НОВОЕ ЗАКЛИНАНИЕ! ===
Добавлено заклинание: Огненная стрела
Заклинаний в руке: 2/5
Счетчик убийств сброшен. До следующего заклинания: 3 убийств
Враг уничтожен! +10 очков
Нажмите любую клавишу для продолжения...

--- Ход врагов ---
Ход врагов завершен. Нажмите любую клавишу...
```

**Вывод:**

В ходе выполнения лабораторной работы была разработана расширенная версия пошаговой игры с использованием объектно-ориентированного подхода.

Были достигнуты следующие результаты:

1. Реализована иерархия классов заклинаний с использованием полиморфизма.
2. Применено правило пяти для классов, владеющих динамическими ресурсами.
3. Реализовано глубокое копирование контейнера заклинаний.
4. Добавлена механика прогрессии игрока (получение новых заклинаний).
5. Реализован расширенный игровой цикл с режимом заклинаний.
6. Обеспечена корректная работа инвариантов классов.
7. Проведено тестирование всех ключевых сценариев.



## Приложение

**main.cpp:**

```
#include "Game.h"
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <conio.h>
#include <windows.h>

int main() {
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    srand(time(NULL));

    try {
        std::cout << "=== Добро пожаловать в игру! ===\n";
        std::cout << "Введите размер поля (ширина высота) от 10 до 25:";
";

        int width, height;
        std::cin >> width >> height;

        Game game(width, height);

        while (game.isRunning()) {
            game.display();

            char key = _getch();
            game.handleInput(key);

            if (game.isRunning() && game.isTurnUsed()) {
                game.enemyTurn();
            }
        }

    } catch (const std::exception& e) {
        std::cout << "Ошибка: " << e.what() << std::endl;
        std::cout << "Нажмите любую клавишу...\n";
    }
}
```

```

        _getch();
    }

    return 0;
}

```

## Cell.cpp:

```

#include "Cell.h"

Cell::Cell() : symbol('.'), hasEnemy(false), hasPlayer(false) {}

Cell::Cell(const Cell& other)
    : symbol(other.symbol), hasEnemy(other.hasEnemy),
    hasPlayer(other.hasPlayer) {}

Cell::Cell(Cell&& other) noexcept
    : symbol(other.symbol), hasEnemy(other.hasEnemy),
    hasPlayer(other.hasPlayer) {
    other.symbol = '.';
    other.hasEnemy = false;
    other.hasPlayer = false;
}

Cell& Cell::operator=(const Cell& other) {
    if (this != &other) {
        symbol = other.symbol;
        hasEnemy = other.hasEnemy;
        hasPlayer = other.hasPlayer;
    }
    return *this;
}

Cell& Cell::operator=(Cell&& other) noexcept {
    if (this != &other) {
        symbol = other.symbol;
        hasEnemy = other.hasEnemy;
        hasPlayer = other.hasPlayer;
        other.symbol = '.';
    }
}

```

```

        other.hasEnemy = false;
        other.hasPlayer = false;
    }
    return *this;
}

char Cell::getSymbol() const { return symbol; }

bool Cell::isEmpty() const {
    return symbol == '.' && !hasEnemy && !hasPlayer;
}

bool Cell::isEnemy() const { return hasEnemy; }
bool Cell::isPlayer() const { return hasPlayer; }

void Cell::setEnemy(bool present) {
    hasEnemy = present;
    if (present) symbol = 'E';
    else if (!hasPlayer) symbol = '.';
}

void Cell::setPlayer(bool present) {
    hasPlayer = present;
    if (present) symbol = 'P';
    else if (!hasEnemy) symbol = '.';
}

```

## Enemy.cpp:

```

#include "Enemy.h"

Enemy::Enemy(int startX, int startY)
    : health(30), damage(10), x(startX), y(startY), alive(true) {}

Enemy::Enemy(const Enemy& other)
    : health(other.health), damage(other.damage),
      x(other.x), y(other.y), alive(other.alive) {}

Enemy::Enemy(Enemy&& other) noexcept
    : health(other.health), damage(other.damage),

```

```

        x(other.x), y(other.y), alive(other.alive) {
other.alive = false;
other.x = -1;
other.y = -1;
}

```

```

Enemy& Enemy::operator=(const Enemy& other) {
    if (this != &other) {
        health = other.health;
        damage = other.damage;
        x = other.x;
        y = other.y;
        alive = other.alive;
    }
    return *this;
}

```

```

Enemy& Enemy::operator=(Enemy&& other) noexcept {
    if (this != &other) {
        health = other.health;
        damage = other.damage;
        x = other.x;
        y = other.y;
        alive = other.alive;
        other.alive = false;
        other.x = -1;
        other.y = -1;
    }
    return *this;
}

```

```

int Enemy::getX() const { return x; }
int Enemy::getY() const { return y; }
bool Enemy::isAlive() const { return alive; }
int Enemy::getDamage() const { return damage; }
int Enemy::getHealth() const { return health; }

```

```

void Enemy::setPosition(int newX, int newY) {

```

```

        x = newX;
        y = newY;
    }

void Enemy::takeDamage(int amount) {
    health -= amount;
    if (health <= 0) {
        alive = false;
    }
}

void Enemy::moveTowardsPlayer(int playerX, int playerY) {
    if (!alive) return;

    if (playerX > x) x++;
    else if (playerX < x) x--;

    if (playerY > y) y++;
    else if (playerY < y) y--;
}

```

### **Game.cpp:**

```

#include "Game.h"
#include <iostream>
#include <cstdlib>
#include <conio.h>

Game::Game(int width, int height)
    : field(width, height),
      player(width/2, height/2),
      running(true),
      spellMode(false),
      selectingTarget(false),
      selectedSpell(-1),
      targetX(0), targetY(0),
      turnUsed(false) {

    field.placePlayer(player.getX(), player.getY());
}

```

```

        for (int i = 0; i < 3; i++) {
            spawnEnemy();
        }
    }

void Game::spawnEnemy() {
    int x, y;
    do {
        x = rand() % field.getWidth();
        y = rand() % field.getHeight();
    } while (!field.isCellEmpty(x, y));

    enemies.push_back(Enemy(x, y));
    field.placeEnemy(x, y);
}

void Game::display() {
    system("cls");
    field.display();

    std::cout << "\n=== Игрок ===\n";
    std::cout << "Здоровье: " << player.getHealth()
        << " | Урон: " << player.getDamage()
        << " | Очки: " << player.getScore() << "\n";
    std::cout << "Врагов осталось: " << getAliveEnemiesCount() <<
"\n";
    std::cout << "Прогресс убийств: " << player.getCurrentKills()
        << "/" << player.getKillsNeeded() << "\n";

    player.getHand()->display();

    if (spellMode) {
        std::cout << "\n=== РЕЖИМ ЗАКЛИНАНИЙ ===\n";
        if (selectingTarget) {
            std::cout << "Цель: (" << targetX << "," << targetY <<
"\n";

            std::cout << "Перемещение: w/a/s/d, Enter - применить
(потратит ход), ESC - отмена\n";

```

```

        } else {
            std::cout << "Выберите заклинание (1-" <<
player.getHand()->getCount()
            << ") или ESC для выхода из режима\n";
        }
    } else {
        std::cout << "\nКоманды: w/a/s/d - движение, f - открыть
заклинания, q - выход\n";
    }

    if (turnUsed) {
        std::cout << "Ход использован! Ожидайте хода врагов...\n";
    }
}

int Game::getAliveEnemiesCount() {
    int count = 0;
    for (auto& enemy : enemies) {
        if (enemy.isAlive()) count++;
    }
    return count;
}

void Game::handleInput(char key) {
    if (turnUsed) {
        return;
    }

    if (spellMode) {
        handleSpellInput(key);
        return;
    }

    switch(key) {
        case 'w': case 'a': case 's': case 'd':
            handleMovement(key);
            break;
        case 'f':

```

```

        enterSpellMode();
        break;
    case 'q':
        running = false;
        break;
    }
}

void Game::handleMovement(char key) {
    int newX = player.getX();
    int newY = player.getY();

    switch(key) {
        case 'w': newY--; break;
        case 's': newY++; break;
        case 'a': newX--; break;
        case 'd': newX++; break;
    }

    if (!field.isValidPosition(newX, newY)) {
        std::cout << "Нельзя выйти за границы поля!\n";
        std::cout << "Нажмите любую клавишу для продолжения...\n";
        _getch();
        return;
    }

    if (field.isCellEnemy(newX, newY)) {
        attackEnemyAt(newX, newY);
        turnUsed = true;
    }
    else if (field.isCellEmpty(newX, newY)) {
        field.clearCell(player.getX(), player.getY());
        player.setPosition(newX, newY);
        field.placePlayer(player.getX(), player.getY());
        turnUsed = true;
    }
}

```



```

void Game::attackEnemyAt(int x, int y) {
    for (auto& enemy : enemies) {
        if (enemy.isAlive() && enemy.getX() == x && enemy.getY() == y)
        {
            std::cout << "Атака! Нанесен урон врагу!\n";
            enemy.takeDamage(player.getDamage());
            if (!enemy.isAlive()) {
                field.clearCell(x, y);
                player.addScore(10);
                std::cout << "Враг уничтожен! +10 очков\n";

                if (getAliveEnemiesCount() == 0) {
                    std::cout << "\n=== ВЫ ПОБЕДИЛИ! Все враги
уничтожены! ===\n";
                    std::cout << "Игра завершена. Нажмите любую
клавишу...\n";

                    _getch();
                    running = false;
                    return;
                }
            }
            std::cout << "Нажмите любую клавишу для продолжения...\n";
            _getch();
            break;
        }
    }
}

```

```

void Game::enterSpellMode() {
    if (player.getHand()->getCount() == 0) {
        std::cout << "Нет заклинаний для использования!\n";
        std::cout << "Нажмите любую клавишу для продолжения...\n";
        _getch();
        return;
    }
    spellMode = true;
    selectingTarget = false;
    selectedSpell = -1;
}

```

```

}

void Game::exitSpellMode() {
    spellMode = false;
    selectingTarget = false;
    selectedSpell = -1;
}

void Game::handleSpellInput(char key) {
    if (!selectingTarget) {
        if (key >= '1' && key <= '9') {
            int index = key - '0';
            if (index <= player.getHand()->getCount()) {
                selectedSpell = index;
                selectingTarget = true;
                targetX = player.getX();
                targetY = player.getY();
            }
        }
        else if (key == 27) {
            exitSpellMode();
        }
    }
    else {
        switch(key) {
            case 'w': targetY--; break;
            case 's': targetY++; break;
            case 'a': targetX--; break;
            case 'd': targetX++; break;
            case 13:
                if (player.getHand()->useSpell(selectedSpell - 1,
targetX, targetY,
field, enemies, player))
{
                turnUsed = true;

                if (getAliveEnemiesCount() == 0) {
                    std::cout << "\n=== ВЫ ПОБЕДИЛИ! Все враги

```

```

уничтожены! ==="\n";

std::cout << "Игра завершена. Нажмите любую
клавишу...\n";

    _getch();
    running = false;
    return;
}
}
exitSpellMode();
break;
case 27:
    selectingTarget = false;
    break;
}
}
}

```

```

void Game::enemyTurn() {
    std::cout << "\n--- Ход врагов ---\n";

    for (auto& enemy : enemies) {
        if (!enemy.isAlive()) continue;

        int oldX = enemy.getX();
        int oldY = enemy.getY();

        enemy.moveTowardsPlayer(player.getX(), player.getY());

        int newX = enemy.getX();
        int newY = enemy.getY();

        if (newX == player.getX() && newY == player.getY()) {
            player.takeDamage(enemy.getDamage());
            enemy.setPosition(oldX, oldY);
        }
        else if (field.isCellEmpty(newX, newY)) {
            field.clearCell(oldX, oldY);
            field.placeEnemy(newX, newY);
        }
    }
}

```

```

        } else {
            enemy.setPosition(oldX, oldY);
        }
    }

    std::cout << "Ход врагов завершен. Нажмите любую клавишу...\n";
    _getch();

    player.resetTurn();
    turnUsed = false;
}

bool Game::isRunning() {
    if (!player.isAlive()) {
        std::cout << "\n=== ИГРОК ПОГИБ ===\n";
        std::cout << "Финальный счет: " << player.getScore() <<
std::endl;
        std::cout << "Нажмите любую клавишу для выхода...\n";
        _getch();
        return false;
    }

    if (getAliveEnemiesCount() == 0 && running) {
        std::cout << "\n=== ВЫ ПОБЕДИЛИ! Все враги уничтожены! ===\n";
        std::cout << "Игра завершена. Нажмите любую клавишу...\n";
        _getch();
        return false;
    }

    return running;
}

bool Game::isTurnUsed() const { return turnUsed; }

```

### Gamefield.cpp:

```

#include "GameField.h"
#include <iostream>
#include <stdexcept>

```

```

GameField::GameField(int w, int h) : width(w), height(h) {
    if (w < 10 || w > 25 || h < 10 || h > 25) {
        throw std::invalid_argument("Размер поля должен быть от 10 до
25");
    }
    grid.resize(height, std::vector<Cell>(width));
}

GameField::GameField(const GameField& other)
    : width(other.width), height(other.height), grid(other.grid) {}

GameField::GameField(GameField&& other) noexcept
    : width(other.width), height(other.height),
grid(std::move(other.grid)) {
    other.width = 0;
    other.height = 0;
}

GameField& GameField::operator=(const GameField& other) {
    if (this != &other) {
        width = other.width;
        height = other.height;
        grid = other.grid;
    }
    return *this;
}

GameField& GameField::operator=(GameField&& other) noexcept {
    if (this != &other) {
        width = other.width;
        height = other.height;
        grid = std::move(other.grid);
        other.width = 0;
        other.height = 0;
    }
    return *this;
}

```

```

int GameField::getWidth() const { return width; }
int GameField::getHeight() const { return height; }

bool GameField::isValidPosition(int x, int y) const {
    return x >= 0 && x < width && y >= 0 && y < height;
}

void GameField::placePlayer(int x, int y) {
    if (isValidPosition(x, y)) {
        grid[y][x].setPlayer(true);
    }
}

void GameField::placeEnemy(int x, int y) {
    if (isValidPosition(x, y)) {
        grid[y][x].setEnemy(true);
    }
}

void GameField::clearCell(int x, int y) {
    if (isValidPosition(x, y)) {
        grid[y][x].setPlayer(false);
        grid[y][x].setEnemy(false);
    }
}

bool GameField::isCellEmpty(int x, int y) const {
    if (!isValidPosition(x, y)) return false;
    return grid[y][x].isEmpty();
}

bool GameField::isCellEnemy(int x, int y) const {
    if (!isValidPosition(x, y)) return false;
    return grid[y][x].isEnemy();
}

void GameField::display() const {

```

```

std::cout << "\n ";
for (int x = 0; x < width; x++) {
    std::cout << x % 10 << " ";
}
std::cout << "\n";

for (int y = 0; y < height; y++) {
    std::cout << y % 10 << " ";
    for (int x = 0; x < width; x++) {
        std::cout << grid[y][x].getSymbol() << " ";
    }
    std::cout << "\n";
}
}

```

### Cell.h:

```

#ifndef CELL_H
#define CELL_H

class Cell {
private:
    char symbol;
    bool hasEnemy;
    bool hasPlayer;

public:
    Cell();
    Cell(const Cell& other);
    Cell(Cell&& other) noexcept;
    Cell& operator=(const Cell& other);
    Cell& operator=(Cell&& other) noexcept;

    char getSymbol() const;
    bool isEmpty() const;
    bool isEnemy() const;
    bool isPlayer() const;

    void setEnemy(bool present);
    void setPlayer(bool present);

```

```
};
```

```
#endif
```

### Enemy.h:

```
#ifndef ENEMY_H
```

```
#define ENEMY_H
```

```
class Enemy {
```

```
private:
```

```
    int health;
```

```
    int damage;
```

```
    int x, y;
```

```
    bool alive;
```

```
public:
```

```
    Enemy(int startX, int startY);
```

```
    Enemy(const Enemy& other);
```

```
    Enemy(Enemy&& other) noexcept;
```

```
    Enemy& operator=(const Enemy& other);
```

```
    Enemy& operator=(Enemy&& other) noexcept;
```

```
    int getX() const;
```

```
    int getY() const;
```

```
    bool isAlive() const;
```

```
    int getDamage() const;
```

```
    int getHealth() const;
```

```
    void setPosition(int newX, int newY);
```

```
    void takeDamage(int amount);
```

```
    void moveTowardsPlayer(int playerX, int playerY);
```

```
};
```

```
#endif
```

### Game.h:

```
#ifndef GAME_H
```

```
#define GAME_H
```

```
#include "GameField.h"
```



```

#include "Player.h"
#include "Enemy.h"
#include <vector>

class Game {
private:
    GameField field;
    Player player;
    std::vector<Enemy> enemies;
    bool running;
    bool spellMode;
    bool selectingTarget;
    int selectedSpell;
    int targetX, targetY;
    bool turnUsed;

public:
    Game(int width, int height);

    void display();
    void handleInput(char key);
    void enemyTurn();
    bool isRunning();
    bool isTurnUsed() const;

private:
    void spawnEnemy();
    int getAliveEnemiesCount();
    void handleMovement(char key);
    void attackEnemyAt(int x, int y);
    void enterSpellMode();
    void exitSpellMode();
    void handleSpellInput(char key);
};

#endif

Gamefield.h:
#ifndef GAME_FIELD_H

```

```

#define GAME_FIELD_H

#include "Cell.h"
#include <vector>

class GameField {
private:
    int width;
    int height;
    std::vector<std::vector<Cell>> grid;

public:
    GameField(int w, int h);
    GameField(const GameField& other);
    GameField(GameField&& other) noexcept;
    GameField& operator=(const GameField& other);
    GameField& operator=(GameField&& other) noexcept;

    int getWidth() const;
    int getHeight() const;
    bool isValidPosition(int x, int y) const;

    void placePlayer(int x, int y);
    void placeEnemy(int x, int y);
    void clearCell(int x, int y);

    bool isEmptyCell(int x, int y) const;
    bool isEnemyCell(int x, int y) const;

    void display() const;
};

#endif

```

### spell.h:

```

#ifndef SPELL_H
#define SPELL_H

```

```

#include <string>
#include <vector>
#include "GameField.h"

class Player;
class Enemy;

class Spell {
protected:
    std::string name;
    int range;
    bool usedThisTurn;

public:
    Spell(const std::string& n, int r);
    virtual ~Spell();

    virtual void use(int targetX, int targetY, GameField& field,
                    std::vector<Enemy>& enemies, Player& player) = 0;

    virtual std::string getDescription() const;

    std::string getName() const;
    int getRange() const;
    bool isUsed() const;
    void setUsed(bool used);

    bool canUse(int playerX, int playerY, int targetX, int targetY)
const;
};

#endif

```

### Spell.cpp:

```

#include "Spell.h"
#include <cmath>

Spell::Spell(const std::string& n, int r) : name(n), range(r),

```

```

usedThisTurn(false) {}

Spell::~~Spell() {}

std::string Spell::getDescription() const {
    return name + " (радиус: " + std::to_string(range) + ")";
}

std::string Spell::getName() const { return name; }
int Spell::getRange() const { return range; }
bool Spell::isUsed() const { return usedThisTurn; }
void Spell::setUsed(bool used) { usedThisTurn = used; }

bool Spell::canUse(int playerX, int playerY, int targetX, int targetY)
const {
    int distance = abs(playerX - targetX) + abs(playerY - targetY);
    return distance <= range;
}

```

### Damagespell.h:

```

#ifndef DAMAGE_SPELL_H
#define DAMAGE_SPELL_H

#include "Spell.h"

class DamageSpell : public Spell {
private:
    int damage;

public:
    DamageSpell();
    void use(int targetX, int targetY, GameField& field,
        std::vector<Enemy>& enemies, Player& player) override;
};

#endif

```

### Damagespell.cpp:

```

#include "DamageSpell.h"

```

```

#include "Player.h"
#include <iostream>
#include <conio.h>

DamageSpell::DamageSpell() : Spell("Огненная стрела", 3), damage(25)
{}

void DamageSpell::use(int targetX, int targetY, GameField& field,
                     std::vector<Enemy>& enemies, Player& player) {
    std::cout << "\n=== ПРИМЕНЕНИЕ ЗАКЛИНАНИЯ ===\n";
    std::cout << "Заклинание: " << name << "\n";
    std::cout << "Цель: (" << targetX << ", " << targetY << ")\n";

    if (!field.isCellEnemy(targetX, targetY)) {
        std::cout << "Нет врага в целевой клетке! Заклинание не
сработало.\n";
        std::cout << "Нажмите любую клавишу для продолжения...\n";
        _getch();
        return;
    }

    for (auto& enemy : enemies) {
        if (enemy.isAlive() && enemy.getX() == targetX && enemy.getY()
== targetY) {
            int oldHealth = enemy.getHealth();
            std::cout << "Враг найден! Текущее здоровье: " <<
oldHealth << "\n";
            std::cout << "Наносится " << damage << " урона!\n";

            enemy.takeDamage(damage);

            if (!enemy.isAlive()) {
                std::cout << "Враг уничтожен! +10 очков\n";
                field.clearCell(targetX, targetY);
                player.addScore(10);
            } else {
                std::cout << "У врага осталось здоровья: " <<
enemy.getHealth() << "\n";

```

```

        }
        break;
    }
}

usedThisTurn = true;
std::cout << "=== ЗАКЛИНАНИЕ ПРИМЕНЕНО ===\n";
std::cout << "Нажмите любую клавишу для продолжения...\n";
_getch();
}

```

### Areaspell.h:

```

#ifndef AREA_SPELL_H
#define AREA_SPELL_H

#include "Spell.h"

class AreaSpell : public Spell {
private:
    int damage;
    int areaSize;

public:
    AreaSpell();
    void use(int targetX, int targetY, GameField& field,
            std::vector<Enemy>& enemies, Player& player) override;
};

#endif

```

### Areaspell.cpp:

```

#include "AreaSpell.h"
#include "Player.h"
#include <iostream>
#include <conio.h>

AreaSpell::AreaSpell() : Spell("Взрывная волна", 4), damage(15),
areaSize(2) {}

```

```

void AreaSpell::use(int targetX, int targetY, GameField& field,
                    std::vector<Enemy>& enemies, Player& player) {
    std::cout << "\n=== ПРИМЕНЕНИЕ ЗАКЛИНАНИЯ ===\n";
    std::cout << "Заклинание: " << name << "\n";
    std::cout << "Центр области: (" << targetX << "," << targetY <<
    ") \n";
    std::cout << "Область поражения: " << areaSize << "x" << areaSize
    << "\n";

    int enemiesHit = 0;
    int totalDamage = 0;

    for (int dx = 0; dx < areaSize; dx++) {
        for (int dy = 0; dy < areaSize; dy++) {
            int checkX = targetX + dx;
            int checkY = targetY + dy;

            if (!field.isValidPosition(checkX, checkY)) continue;

            for (auto& enemy : enemies) {
                if (enemy.isAlive() && enemy.getX() == checkX &&
enemy.getY() == checkY) {
                    enemiesHit++;
                    totalDamage += damage;

                    std::cout << "Враг в клетке (" << checkX << "," <<
checkY
                        << ") получил " << damage << " урона!\n";

                    enemy.takeDamage(damage);

                    if (!enemy.isAlive()) {
                        std::cout << " Враг уничтожен!\n";
                        field.clearCell(checkX, checkY);
                        player.addScore(10);
                    }
                    break;
                }
            }
        }
    }
}

```

```

        }

    }

}

if (enemiesHit == 0) {
    std::cout << "В области нет врагов. Заклинание потрачено
впустую.\n";
} else {
    std::cout << "Всего поражено врагов: " << enemiesHit << "\n";
    std::cout << "Суммарный урон: " << totalDamage << "\n";
}

usedThisTurn = true;
std::cout << "=== ЗАКЛИНАНИЕ ПРИМЕНЕНО ===\n";
std::cout << "Нажмите любую клавишу для продолжения...\n";
_getch();
}

```

## Hand.h:

```

#ifndef HAND_H
#define HAND_H

#include "Spell.h"
#include <vector>

class Hand {
private:
    std::vector<Spell*> spells;
    int maxSize;

    bool hasSpellOfType(int type);

public:
    Hand(int size);
    Hand(const Hand& other);
    Hand(Hand&& other) noexcept;
    Hand& operator=(const Hand& other);
    Hand& operator=(Hand&& other) noexcept;

```



```

~Hand();

bool addSpell(Spell* spell);
void addRandomSpell();
bool useSpell(int index, int targetX, int targetY,
              GameField& field, std::vector<Enemy>& enemies,
Player& player);
    void display() const;
    void resetTurn();
    int getCount() const;
};

#endif

```

### Hand.cpp:

```

#include "Hand.h"
#include "DamageSpell.h"
#include "AreaSpell.h"
#include "Player.h"
#include <iostream>
#include <cstdlib>
#include <conio.h>

Hand::Hand(int size) : maxSize(size) {
    addRandomSpell();
}

Hand::Hand(const Hand& other) : maxSize(other.maxSize) {
    for (Spell* spell : other.spells) {
        if (dynamic_cast<DamageSpell*>(spell)) {
            spells.push_back(new
DamageSpell(*dynamic_cast<DamageSpell*>(spell)));
        } else if (dynamic_cast<AreaSpell*>(spell)) {
            spells.push_back(new
AreaSpell(*dynamic_cast<AreaSpell*>(spell)));
        }
    }
}

```

```

Hand& Hand::operator=(const Hand& other) {
    if (this != &other) {
        for (Spell* spell : spells) {
            delete spell;
        }
        spells.clear();

        maxSize = other.maxSize;
        for (Spell* spell : other.spells) {
            if (dynamic_cast<DamageSpell*>(spell)) {
                spells.push_back(new
DamageSpell(*dynamic_cast<DamageSpell*>(spell)));
            } else if (dynamic_cast<AreaSpell*>(spell)) {
                spells.push_back(new
AreaSpell(*dynamic_cast<AreaSpell*>(spell)));
            }
        }
    }
    return *this;
}

Hand::Hand(Hand&& other) noexcept
    : spells(std::move(other.spells)), maxSize(other.maxSize) {}

Hand& Hand::operator=(Hand&& other) noexcept {
    if (this != &other) {
        spells = std::move(other.spells);
        maxSize = other.maxSize;
    }
    return *this;
}

Hand::~~Hand() {
    for (Spell* spell : spells) {
        delete spell;
    }
}

```

```

bool Hand::hasSpellOfType(int type) {
    for (Spell* spell : spells) {
        if (type == 0 && dynamic_cast<DamageSpell*>(spell)) return
true;
        if (type == 1 && dynamic_cast<AreaSpell*>(spell)) return true;
    }
    return false;
}

bool Hand::addSpell(Spell* spell) {
    if (spells.size() >= maxSize) {
        std::cout << "Рука полна! Нельзя добавить новое
заклинание.\n";
        delete spell;
        return false;
    }

    spells.push_back(spell);
    std::cout << "Добавлено заклинание: " << spell->getName() << "\n";
    std::cout << "Заклинаний в руке: " << spells.size() << "/" <<
maxSize << "\n";
    return true;
}

void Hand::addRandomSpell() {
    if (spells.size() >= maxSize) return;

    if (spells.size() == 1) {
        bool hasDamage = hasSpellOfType(0);
        if (hasDamage) {
            addSpell(new AreaSpell());
            return;
        } else {
            addSpell(new DamageSpell());
            return;
        }
    }
}

```

```

int type = rand() % 2;
if (type == 0) {
    addSpell(new DamageSpell());
} else {
    addSpell(new AreaSpell());
}
}

bool Hand::useSpell(int index, int targetX, int targetY,
                    GameField& field, std::vector<Enemy>& enemies,
Player& player) {
    if (index < 0 || index >= spells.size()) {
        std::cout << "Неверный индекс заклинания!\n";
        std::cout << "Нажмите любую клавишу для продолжения...\n";
        _getch();
        return false;
    }

    Spell* spell = spells[index];

    if (spell->isUsed()) {
        std::cout << "Это заклинание уже использовано в этот ход!\n";
        std::cout << "Нажмите любую клавишу для продолжения...\n";
        _getch();
        return false;
    }

    int playerX = player.getX();
    int playerY = player.getY();

    std::cout << "\nПроверка дальности:\n";
    std::cout << "Игрок: (" << playerX << "," << playerY << ")\n";
    std::cout << "Цель: (" << targetX << "," << targetY << ")\n";

    if (!spell->canUse(playerX, playerY, targetX, targetY)) {
        int distance = abs(playerX - targetX) + abs(playerY -
targetY);

```

```

        std::cout << "Цель вне радиуса! Расстояние: " << distance
            << ", радиус заклинания: " << spell->getRange() << "\n";
        std::cout << "Нажмите любую клавишу для продолжения...\n";
        _getch();
        return false;
    }

    std::cout << "Цель в радиусе действия\n";

    spell->use(targetX, targetY, field, enemies, player);
    return true;
}

void Hand::display() const {
    if (spells.empty()) {
        std::cout << "Рука пуста.\n";
        return;
    }

    std::cout << "\n--- Рука игрока ---\n";
    for (size_t i = 0; i < spells.size(); i++) {
        std::cout << i + 1 << ". " << spells[i]->getDescription();
        if (spells[i]->isUsed()) {
            std::cout << " [ИСПОЛЬЗОВАНО]";
        }
        std::cout << "\n";
    }
}

void Hand::resetTurn() {
    for (auto spell : spells) {
        spell->setUsed(false);
    }
}

int Hand::getCount() const { return spells.size(); }

```

**Player.h:**

```

#ifndef PLAYER_H
#define PLAYER_H

#include "Hand.h"

class Player {
private:
    int health;
    int damage;
    int score;
    int x, y;
    Hand* hand;
    int killsForNewSpell;
    int currentKills;

public:
    Player(int startX, int startY);
    Player(const Player& other);
    Player(Player&& other) noexcept;
    Player& operator=(const Player& other);
    Player& operator=(Player&& other) noexcept;
    ~Player();

    int getX() const;
    int getY() const;
    int getHealth() const;
    int getDamage() const;
    int getScore() const;
    Hand* getHand() const;
    int getCurrentKills() const;
    int getKillsNeeded() const;

    void setPosition(int newX, int newY);
    void takeDamage(int amount);
    void addScore(int points);
    bool isAlive() const;

    void onEnemyKilled();

```

```

        void resetTurn();
    };

#endif

```

### Player.cpp:

```

#include "Player.h"
#include <iostream>

Player::Player(int startX, int startY)
    : health(100), damage(10), score(0), x(startX), y(startY),
      hand(new Hand(5)), killsForNewSpell(3), currentKills(0) {}

Player::Player(const Player& other)
    : health(other.health), damage(other.damage),
      score(other.score), x(other.x), y(other.y),
      hand(new Hand(*other.hand)),
      killsForNewSpell(other.killsForNewSpell),
      currentKills(other.currentKills) {}

Player::Player(Player&& other) noexcept
    : health(other.health), damage(other.damage),
      score(other.score), x(other.x), y(other.y),
      hand(other.hand),
      killsForNewSpell(other.killsForNewSpell),
      currentKills(other.currentKills) {
    other.health = 0;
    other.x = -1;
    other.y = -1;
    other.hand = nullptr;
}

Player& Player::operator=(const Player& other) {
    if (this != &other) {
        health = other.health;
        damage = other.damage;
        score = other.score;
        x = other.x;

```

```

        y = other.y;

        delete hand;
        hand = new Hand(*other.hand);

        killsForNewSpell = other.killsForNewSpell;
        currentKills = other.currentKills;
    }
    return *this;
}

Player& Player::operator=(Player&& other) noexcept {
    if (this != &other) {
        health = other.health;
        damage = other.damage;
        score = other.score;
        x = other.x;
        y = other.y;

        delete hand;
        hand = other.hand;

        killsForNewSpell = other.killsForNewSpell;
        currentKills = other.currentKills;

        other.health = 0;
        other.x = -1;
        other.y = -1;
        other.hand = nullptr;
    }
    return *this;
}

Player::~~Player() {
    delete hand;
}

int Player::getX() const { return x; }

```



```

int Player::getY() const { return y; }
int Player::getHealth() const { return health; }
int Player::getDamage() const { return damage; }
int Player::getScore() const { return score; }
Hand* Player::getHand() const { return hand; }
int Player::getCurrentKills() const { return currentKills; }
int Player::getKillsNeeded() const { return killsForNewSpell; }

void Player::setPosition(int newX, int newY) {
    x = newX;
    y = newY;
}

void Player::takeDamage(int amount) {
    health -= amount;
    if (health < 0) health = 0;
    std::cout << "Игрок получил " << amount << " урона! Здоровье: " <<
health << std::endl;
}

void Player::addScore(int points) {
    score += points;
    std::cout << "Получено очков: " << points << ". Всего: " << score
<< "\n";
    onEnemyKilled();
}

bool Player::isAlive() const { return health > 0; }

void Player::onEnemyKilled() {
    currentKills++;
    std::cout << "Прогресс убийств: " << currentKills << "/" <<
killsForNewSpell << "\n";

    if (currentKills >= killsForNewSpell) {
        std::cout << "=== ПОЛУЧЕНО НОВОЕ ЗАКЛИНАНИЕ! ===\n";
        hand->addRandomSpell();
        currentKills = 0;
    }
}

```

```
        std::cout << "Счетчик убийств сброшен. До следующего  
заклинания: "  
        << killsForNewSpell << " убийств\n";  
    }  
}  
  
void Player::resetTurn() {  
    hand->resetTurn();  
}
```